

PYTHON CHEATSHEET

Complete Revision Reference — Syntax, Concepts & Quick Examples

Basics -> Data Structures -> Functions -> OOP -> NumPy -> Standard Library

1. Basics & Syntax

Comments, printing, input, variables, type checking.

```
# Single-line comment
'''Multi-line
    comment / docstring'''

print("Hello, World!")      # print to console
x = 5; y = 10               # multiple statements
name = input("Your name: ") # read user input (returns str)

a, b, c = 1, 2, 3          # multiple assignment
a, b = b, a                # swap values
type(a)                    # <class 'int'>
isinstance(a, int)         # True
```

2. Variables & Data Types

Core built-in types and conversions.

```
i = 10          # int
f = 3.14        # float
s = "text"      # str
b = True        # bool
n = None        # NoneType
c = 3 + 4j      # complex

# Type conversion
int("42"); float("3.14"); str(99)
bool(0) # False    bool(1) # True
list("abc") # ['a','b','c']

# Check size / identity / membership
len(s); id(x); x is None; 5 in [1,5,9]
```

3. Operators

Arithmetic, comparison, logical, bitwise, assignment.

```
# Arithmetic
+ - * / // % **      # // floor div, ** power
7 // 2   # 3        7 % 2   # 1        2 ** 3   # 8

# Comparison
== != > < >= <=

# Logical
and or not

# Bitwise
& | ^ ~ << >>

# Assignment shorthand
x += 1; x -= 1; x *= 2; x /= 2; x **= 2

# Walrus operator (assign within expression)
if (n := len([1,2,3])) > 2:
    print(n)
```

4. Strings

Indexing, slicing, common methods, f-strings.

```
s = "Hello, Python!"
s[0]      # 'H'          s[-1]      # '!'
s[0:5]    # 'Hello'     s[::-1]    # reversed string

s.upper(); s.lower(); s.strip()
s.replace("Hello", "Hi")
s.split(","); ", ".join(["a", "b"])
s.find("Py"); s.startswith("He"); s.endswith("!")
s.count("l"); len(s)

name, age = "Alia", 30
f"{name} is {age} years old"      # f-string
f"{3.14159:.2f}"                 # '3.14' (format spec)
("{} and {}".format("a", "b"))   # .format()
"%s is %d" % (name, age)         # % formatting
```

5. Lists

Ordered, mutable sequences.

```
lst = [1, 2, 3, 4, 5]
lst.append(6); lst.insert(0, 0); lst.extend([7,8])
lst.remove(3); lst.pop(); lst.pop(0)
lst.sort(); lst.sort(reverse=True); lst.reverse()
lst.index(4); lst.count(2); len(lst)

lst[1:3]          # slice
lst[:3]; lst[2:]; lst[::2]    # step slicing
lst.copy()        # shallow copy

# List comprehension
squares = [x**2 for x in range(10)]
evens = [x for x in range(10) if x % 2 == 0]
pairs = [(x, y) for x in range(3) for y in range(3)]
```

6. Tuples & Sets

Immutable sequences and unique collections.

```
# Tuple - ordered, IMMUTABLE
t = (1, 2, 3)
t[0]          # 1 (indexing works)
a, b, c = t    # unpacking
single = (5,)  # single-element tuple needs comma

# Set - unordered, unique elements
s1 = {1, 2, 3}
s2 = {3, 4, 5}
s1 | s2        # union          s1 & s2    # intersection
s1 - s2        # difference     s1 ^ s2    # symmetric diff
s1.add(4); s1.discard(1); s1.remove(2)

{x for x in range(10) if x % 2 == 0} # set comprehension
frozenset([1, 2, 3])                 # immutable set
```

7. Dictionaries

Key-value pairs (insertion-ordered since 3.7).

```
d = {"name": "Alia", "age": 30}
d["name"]           # access
d.get("city", "N/A") # safe access with default
d["city"] = "Delhi"  # add/update
del d["age"]; d.pop("city")

d.keys(); d.values(); d.items()
for k, v in d.items():
    print(k, v)

{k: v for k, v in d.items()} # dict comprehension
{x: x**2 for x in range(5)}  # build new dict

d1 = {"a": 1}; d2 = {"b": 2}
d1 | d2 # merge (3.9+)
d1.update(d2) # merge in place
```

8. Control Flow

Conditionals, loops, and loop control.

```
if x > 0:
    print("positive")
elif x == 0:
    print("zero")
else:
    print("negative")

for i in range(5):          # 0..4
    if i == 3:
        break
    if i == 1:
        continue
    print(i)

for i, val in enumerate(["a", "b"]): # index + value
    print(i, val)

while x > 0:
    x -= 1

# Match-case (3.10+, like switch)
match x:
    case 0:
        print("zero")
    case 1 | 2:
        print("one or two")
    case _:
        print("other")
```

9. Functions

*Definitions, defaults, *args/**kwargs, lambdas.*

```
def greet(name, greeting="Hello"): # default arg
    return f"{greeting}, {name}!"

def add_all(*args):                # variable positional args
    return sum(args)

def show(**kwargs):                # variable keyword args
    for k, v in kwargs.items():
        print(k, v)

def combo(a, *args, b=1, **kwargs): # mixed
    pass

square = lambda x: x ** 2          # anonymous function
add = lambda a, b: a + b

def outer():                       # closures
    msg = "hi"
    def inner():
        return msg
    return inner
```

10. Comprehensions & Generators

Compact, memory-efficient constructs.

```
[x*2 for x in range(5)]           # list comprehension
{x*2 for x in range(5)}           # set comprehension
{x: x*2 for x in range(5)}        # dict comprehension
(x*2 for x in range(5))           # generator expression (lazy)

def counter(n):
    i = 0
    while i < n:
        yield i                   # generator function
        i += 1

gen = counter(3)
next(gen); next(gen)              # pull values one at a time

it = iter([1, 2, 3])              # manual iterator
next(it)
```

11. Exception Handling

try/except/else/finally, custom exceptions.

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print("Error:", e)
except (TypeError, ValueError):
    print("Type or value issue")
else:
    print("No exception occurred")
finally:
    print("Always runs")

raise ValueError("Invalid input")      # raise manually

class MyError(Exception):              # custom exception
    pass

assert x > 0, "x must be positive"     # assertion
```

12. File Handling

Reading, writing, and working with files.

```
with open("file.txt", "r") as f:      # 'with' auto-closes file
    content = f.read()
    lines = f.readlines()
    for line in f:
        print(line.strip())

with open("file.txt", "w") as f:      # 'w' overwrites
    f.write("Hello\n")

with open("file.txt", "a") as f:      # 'a' appends
    f.write("More text\n")

# Modes: 'r' read, 'w' write, 'a' append,
#        'x' create, 'b' binary, '+' read/write
```

13. OOP - Classes & Objects

Constructors, methods, inheritance, magic methods.

```
class Animal:
    species = "Unknown"          # class attribute

    def __init__(self, name):     # constructor
        self.name = name        # instance attribute

    def speak(self):             # instance method
        return f"{self.name} makes a sound"

    def __str__(self):            # string representation
        return f"Animal({self.name})"

class Dog(Animal):               # inheritance
    def speak(self):             # method override
        return f"{self.name} barks"

    def speak_loud(self):
        return super().speak() + "!!" # call parent method

d = Dog("Rex")
isinstance(d, Animal)           # True
```

14. OOP - Decorators & Properties

@property, @staticmethod, @classmethod.

```
class Circle:
    def __init__(self, radius):
        self._radius = radius

    @property
    def area(self):                # access like an attribute
        return 3.14159 * self._radius ** 2

    @staticmethod
    def is_valid(radius):          # no self/cls needed
        return radius > 0

    @classmethod
    def unit_circle(cls):          # alternate constructor
        return cls(radius=1)

c = Circle(5)
c.area                           # called without ()
Circle.is_valid(5)
Circle.unit_circle()
```


15. Modules & Packages

Importing code and running scripts.

```
import math                # import whole module
from math import sqrt, pi  # import specific names
from math import *         # import everything (avoid)
import numpy as np         # import with alias

# my_module.py
def helper(): ...

# main.py
import my_module
my_module.helper()

if __name__ == "__main__":    # only runs when executed directly
    main()
```

16. Decorators (Functions)

Wrapping functions to extend behavior.

```
import functools

def my_decorator(func):
    @functools.wraps(func)          # preserves func metadata
    def wrapper(*args, **kwargs):
        print("Before call")
        result = func(*args, **kwargs)
        print("After call")
        return result
    return wrapper

@my_decorator
def say_hi():
    print("Hi!")

say_hi()    # "Before call" -> "Hi!" -> "After call"
```

17. Useful Built-in Functions

Frequently used higher-order/utility functions.

```
len([1,2,3]); range(5); range(1,10,2)
sorted([3,1,2]); sorted(d, reverse=True)
sum([1,2,3]); min(1,2); max([4,5,6])
abs(-5); round(3.14159, 2)

map(lambda x: x*2, [1,2,3])          # apply function to all
filter(lambda x: x>2, [1,2,3,4])     # keep matching items
zip([1,2],[3,4])                     # pair elements together
enumerate(["a","b"])                 # (index, value) pairs

any([False, True]); all([True, True])
list(...); tuple(...); set(...); dict(...)
```

18. Common Standard Library Modules

Quick reference to frequently used modules.

```
import os          # os.getcwd(), os.listdir(), os.path.join()
import sys         # sys.argv, sys.exit(), sys.path
import math        # math.sqrt(), math.floor(), math.pi
import random      # random.randint(), random.choice(), random.shuffle()
import datetime    # datetime.now(), timedelta
import json        # json.dumps(), json.loads()
import re          # re.match(), re.search(), re.findall()
import collections # Counter, defaultdict, namedtuple, OrderedDict
import itertools   # permutations, combinations, product, chain
```

19. Virtual Environments & pip

Isolating project dependencies.

```
python -m venv venv          # create virtual environment
source venv/bin/activate     # activate (Linux/Mac)
venv\Scripts\activate       # activate (Windows)

pip install package_name     # install a package
pip install -r requirements.txt # install from file
pip freeze > requirements.txt # export installed packages
pip uninstall package_name   # remove a package
pip list                     # list installed packages
```

20. Common Gotchas & Best Practices

Things that frequently trip up beginners.

```
# Mutable default argument (BUG)
def bad(items=[]): ...           # avoid! shared across calls
def good(items=None):
    items = items or []

# Copying lists
a = [1, 2, 3]
b = a           # same reference! changes to b affect a
b = a.copy()    # independent copy
b = a[:]        # also a copy

# Comparing values vs identity
a == b          # value equality
a is b          # identity (same object in memory)

# f-strings are preferred over % and .format() (Python 3.6+)
```

21. NumPy - Array Creation

Creating 1D, 2D, and 3D arrays.

```
import numpy as np

# From existing data
a1 = np.array([1, 2, 3])           # 1D array, shape (3,)
a2 = np.array([[1, 2], [3, 4]])    # 2D array, shape (2,2)
a3 = np.array([[1,2],[3,4]],[[5,6],[7,8]]) # 3D, shape (2,2,2)

# Built-in generators
np.zeros((3,))                     # 1D zeros: [0. 0. 0.]
np.zeros((2, 3))                   # 2D zeros: shape (2,3)
np.zeros((2, 3, 4))                # 3D zeros: shape (2,3,4)
np.ones((2, 2))                    # array of ones
np.full((2, 2), 7)                 # filled with a value
np.eye(3)                          # 3x3 identity matrix

np.arange(0, 10, 2)                # [0 2 4 6 8] (like range())
np.linspace(0, 1, 5)               # 5 evenly spaced values 0..1

np.random.rand(2, 3)               # uniform random, shape (2,3)
np.random.randint(0, 10, (2,3))    # random ints
np.random.seed(42)                 # reproducible randomness
```

22. NumPy - Array Properties & Indexing

Shape, dtype, and accessing elements (1D/2D/3D).

```
a = np.array([[[1,2,3],[4,5,6]],[[7,8,9],[10,11,12]]])

a.shape      # (2, 2, 3)  -> (depth, rows, cols)
a.ndim       # 3        (number of dimensions)
a.size       # 12       (total elements)
a.dtype      # dtype('int64')
a.astype(float)      # cast to another type

# 1D indexing
v = np.array([10, 20, 30, 40])
v[0]; v[-1]; v[1:3]

# 2D indexing: [row, col]
m = np.array([[1,2,3],[4,5,6]])
m[0, 1]      # 2      (row 0, col 1)
m[1, :]      # entire row 1
m[:, 0]      # entire column 0
m[0:2, 1:3]  # sub-matrix slice

# 3D indexing: [depth, row, col]
a[0]         # first 2D slice (matrix)
a[0, 1]      # row 1 of first matrix -> [4 5 6]
a[0, 1, 2]   # single element -> 6
a[:, 0, :]   # row 0 from every depth slice
```

23. NumPy - Reshaping & Combining Dimensions

Converting between 1D, 2D, and 3D shapes.

```
a = np.arange(12)          # 1D: [0 1 2 ... 11]

a.reshape(3, 4)            # -> 2D, shape (3,4)
a.reshape(2, 3, 2)         # -> 3D, shape (2,3,2)
a.reshape(-1, 4)           # -1 = auto-infer that dimension
a.reshape(3, 4).flatten()  # back to 1D (copy)
a.reshape(3, 4).ravel()    # back to 1D (view if possible)

m = np.array([[1,2],[3,4]])
m.T                        # transpose (swap axes)
m.reshape(2,2).transpose(1,0) # same as .T for 2D

a3 = np.arange(8).reshape(2,2,2)
a3.transpose(1,0,2)        # reorder axes for 3D
np.expand_dims(m, axis=0)   # add new axis -> shape (1,2,2)
np.squeeze(np.zeros((1,3,1))) # remove size-1 axes
```

24. NumPy - Math Operations & Broadcasting

Element-wise math and automatic shape expansion.

```
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])

a + b; a - b; a * b; a / b      # element-wise ops
a ** 2                        # square each element
np.sqrt(a); np.exp(a); np.log(a)
np.sin(a); np.abs(a)

# Broadcasting: smaller shape auto-expands to match
m = np.array([[1,2,3],[4,5,6]]) # shape (2,3)
m + np.array([10, 20, 30])      # adds to every row (3,) -> (2,3)
m + np.array([[1],[2]])         # adds to every col (2,1) -> (2,3)
m * 2                           # scalar broadcasts to all elements

# Matrix operations
np.dot(a, b)                    # dot product
m @ m.T                        # matrix multiplication
np.matmul(m, m.T)              # same as @
```

25. NumPy - Aggregation & Axis Operations

Summarizing data along rows, columns, or depth.

```
m = np.array([[1,2,3],[4,5,6]]) # shape (2,3)

m.sum()                # total sum of all elements -> 21
m.sum(axis=0)          # sum down each column -> [5 7 9]
m.sum(axis=1)          # sum across each row -> [6 15]

m.mean(); m.mean(axis=0) # average (overall / per column)
m.max(); m.min()
m.max(axis=1)           # max in each row
m.argmax(); m.argmin()  # index of max/min value
m.std(); m.var()        # std deviation / variance

a3 = np.arange(24).reshape(2,3,4) # 3D array
a3.sum(axis=0)            # collapse depth -> shape (3,4)
a3.sum(axis=(1,2))        # collapse rows+cols -> shape (2,)

np.sort(m, axis=1)       # sort within each row
m[m > 3]                 # boolean mask filtering
np.where(m > 3, 1, 0)    # conditional replace
```

26. NumPy - Stacking, Splitting & Utilities

Joining and dividing arrays; other handy tools.

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

np.concatenate([a, b])          # join 1D -> [1 2 3 4 5 6]
np.vstack([a, b])               # stack as rows -> shape (2,3)
np.hstack([a, b])               # stack side by side (1D)
np.stack([a, b], axis=0)        # new axis -> shape (2,3)
np.stack([a, b], axis=-1)       # stack as columns -> shape (3,2)

m = np.arange(12).reshape(3, 4)
np.split(m, 3, axis=0)          # split into 3 row-groups
np.hsplit(m, 2)                 # split into 2 column-groups

np.array_equal(a, b)            # compare full arrays
np.unique([1,1,2,3,3])          # distinct values -> [1 2 3]
np.clip(a, 1, 2)                # limit values to a range
a.copy()                        # independent copy (not a view)
np.save("arr.npy", a); np.load("arr.npy") # save/load to disk
```

End of reference - covers core syntax through OOP, file I/O, decorators, the standard library, and NumPy (1D/2D/3D arrays).